Lightweight Object-Level Image Editing on Edge Devices: Segmentation, Removal, and Replacement

***Abstract*—Object-level image editing on edge hardware requires a careful balance between mask accuracy, reconstruction quality, latency, memory, and implementation reliability. This work presents an empirical study of lightweight pipelines for text-guided object removal, with an additional exploration of object replacement under a constrained development budget. Eight segmentation–inpainting architectures were implemented and benchmarked on a ten-image COCO subset, covering MobileSAM, EfficientSAM-S, YOLO26n-seg, Grounding DINO-Tiny combinations, LaMa variants, and MI-GAN. The experiments expose a clear separation between segmentation/grounding cost and inpainting cost: Grounding DINO substantially increases latency, while LaMa remains the dominant reconstruction stage in lightweight pipelines. The strongest measured standalone trade-off was MobileSAM + LaMa, with 90% execution success, 0.810 mean IoU, 0.892 mean Dice, and 16.63 s mean successful latency. EfficientSAM-S + LaMa improved mask quality to 0.827 IoU and 0.903 Dice at 25.74 s, while YOLO26n-seg + LaMa reached 0.847 IoU and 0.916 Dice on successful runs but exposed mask-shape and detection failures. MI-GAN could not be fairly evaluated because every run failed at the ONNX interface due to a mask-rank mismatch. A subsequent single-image edge-oriented test measured 6.926 s for LaMa and only 0.006 s for replacement compositing, confirming that reconstruction, rather than compositing, is the principal optimization target. Object replacement was explored through lightweight compositing and generative replacement concepts, but extensive benchmarking was not completed because of time constraints.**

***Index Terms*—edge AI, object removal, image inpainting, semantic segmentation, MobileSAM, EfficientSAM, YOLO, Grounding DINO, LaMa, MI-GAN, object replacement**

I. Introduction

Automated photo editing is increasingly shifting from manual region selection toward intent-driven editing, where a user specifies an operation and the system identifies the relevant region, modifies it, and returns a visually coherent result. The practical difficulty is not a single model: an editing system must chain localization, segmentation, image reconstruction, and compositing while remaining robust to failures at the interfaces between stages.

The design target in this study is edge-oriented inference. The reference design emphasizes accurate masks, CPU-friendly models, ONNX deployment, and lightweight generative processing; it also describes MobileSAM, MobileCLIP, LaMa, Stable Diffusion 1.5 with LCM, and an agentic routing layer as components of a broader automated editing system. [9]

This report concentrates on the experimentally developed object-removal path. The study compares eight concrete architectures recorded in the benchmark artifact and documents the failures that prevented several seemingly promising alternatives from becoming reliable end-to-end systems. The report also records the limited object-replacement exploration and explicitly separates demonstrated measurements from approaches that were only partially trialed.



Fig. 1. Experimental object-removal architecture. The study varied the grounding and segmentation blocks while evaluating LaMa and MI-GAN as reconstruction alternatives.

II. System Objective and Design Constraints

The target workflow is

$$I, P \rightarrow \text{localization} \rightarrow M \rightarrow \text{inpainting} \rightarrow \hat{I}, \qquad (1)$$

where I is the input image, P is the user instruction, M is the selected object mask, and $\hat{I}$ is the edited image.

The principal constraints were:

- CPU-oriented or edge-compatible inference;
- low memory footprint and ONNX-compatible components where possible;
- accurate object masks for downstream editing;
- visually coherent object removal;
- measurable per-stage latency rather than only end-to-end timing;
- graceful handling of detection, mask, and model-interface failures.

III. Architectures Explored

A. MobileSAM + LaMa

MobileSAM was selected as the primary lightweight SAM-family baseline. It retains a SAM-compatible prompting interface while replacing the heavy vision encoder with a Tiny-ViT-style backbone. The measured pipeline achieved 90% successful executions, 0.810 mean IoU, and 0.892 mean Dice, with 4.34 s segmentation/preparation time and 12.29 s LaMa time on successful runs. Its 16.63 s mean successful latency made it the fastest reliable pipeline among the tested configurations. The reference document likewise identifies

MobileSAM as a practical mobile segmentation backbone because of its substantially lower compute relative to standard SAM variants. [1], [2], [9]

B. EfficientSAM-S + LaMa

EfficientSAM-S was evaluated as a stronger lightweight segmentation alternative. It produced 0.827 mean IoU and 0.903 mean Dice with a 90% execution success rate. Segmentation increased to 13.29 s while LaMa remained approximately 12.45 s, giving 25.74 s mean successful latency. The result demonstrates a quality–latency trade-off: mask quality improved over MobileSAM, but the segmentation cost increased substantially.

C. YOLO26n-seg + LaMa

YOLO26n-seg was evaluated as a compact one-stage segmentation route. On successful runs it reached 0.847 mean IoU and 0.916 mean Dice, the highest mask-quality values in the benchmark. However, only six of ten runs completed successfully. The failures included incompatible image/mask shapes such as $(427, 640, 3)$ versus $(448, 640, 1)$ and $(408, 640, 3)$ versus $(416, 640, 1)$, as well as a no-target case. Thus, the high quality values should be interpreted together with the 60% execution success rate rather than as an unconditional pipeline result.

D. Grounding DINO-Tiny + YOLO26n-seg + LaMa

Grounding DINO-Tiny was added to test text-guided localization before YOLO segmentation. The successful subset retained 0.847 mean IoU and 0.916 mean Dice, but grounding/segmentation increased to 33.66 s and total successful latency to 46.06 s. The result shows the cost of explicit text grounding: the mask quality was not improved over the standalone YOLO configuration, while latency increased markedly.

E. Grounding DINO-Tiny + MobileSAM + LaMa

This architecture combines text grounding with MobileSAM refinement. All ten images completed successfully, with 0.805 mean IoU and 0.885 mean Dice. The grounding-plus-segmentation stage averaged 38.05 s and the complete successful pipeline averaged 50.32 s. Compared with standalone MobileSAM, text grounding increased latency substantially without producing a corresponding increase in measured mask quality.

F. Grounding DINO-Tiny + EfficientSAM-S + LaMa

The complete grounding–segmentation pipeline achieved 100% execution success, 0.814 mean IoU, and 0.891 mean Dice. Its average grounding/segmentation stage required 49.85 s, followed by 12.81 s of LaMa, for 62.66 s total. This configuration is robust in execution but expensive for edge deployment.

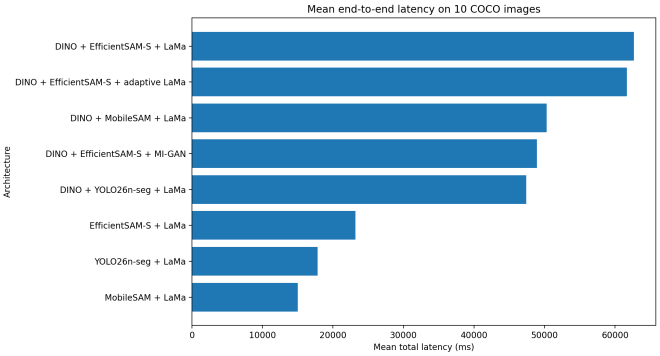


Fig. 2. Mean total latency reported by the benchmark artifact. Lower latency is preferable for edge deployment, but success rate and mask quality must be considered jointly.

G. Adaptive LaMa Variant

An adaptive LaMa variant was tested with the same DINO + EfficientSAM-S front end. It preserved the same measured IoU and Dice values while reducing mean successful latency from 62.66 s to 61.69 s. The reduction was therefore approximately 1.0 s at the pipeline level, indicating that segmentation/grounding, rather than LaMa adaptation, dominated the runtime in this configuration.

H. Grounding DINO-Tiny + EfficientSAM-S + MI-GAN

MI-GAN was tested as an alternative inpainting backend. All ten runs failed before producing a valid reconstruction because ONNX Runtime reported an invalid mask rank: the supplied mask had rank 3 while the model expected rank 4. Consequently, no MI-GAN quality value is reported. This is an implementation-interface failure, not evidence that MI-GAN produces poor visual quality.

IV. Experimental Protocol

The benchmark contains eight architectures evaluated over ten COCO images, yielding 80 architecture–image records. Each record stores segmentation time, grounding score where applicable, inpainting time, total time, mask area, change score, IoU, Dice, success status, and error information.

For successful samples, mask quality is summarized by intersection-over-union and Dice coefficient:

$$\mathrm{IoU} = \frac{|M_p \cap M_g|}{|M_p \cup M_g|}, \tag{2}$$

$$\mathrm{Dice} = \frac{2|M_p \cap M_g|}{|M_p| + |M_g|}. \tag{3}$$

Here M_p and M_g denote predicted and reference masks. Latency is decomposed into segmentation/grounding and inpainting components. Execution success rate is defined as the fraction of the ten attempted images that completed without a recorded pipeline exception.

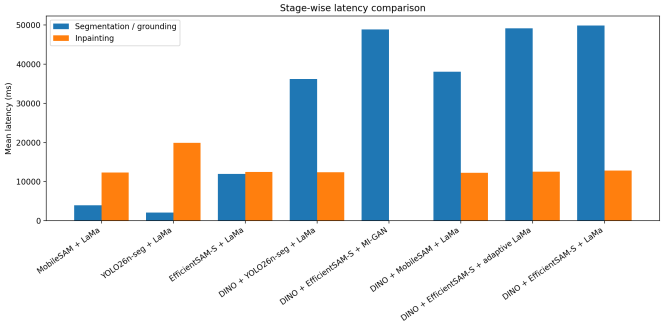


Fig. 3. Stage-wise latency comparison. Grounding-heavy configurations spend most of their runtime before inpainting, while LaMa dominates the lightweight standalone pipelines.

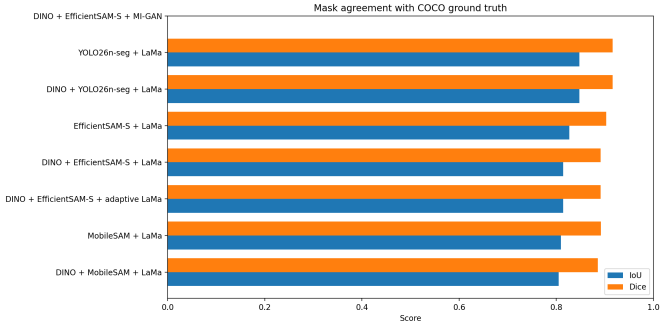


Fig. 4. Mask quality across the tested architectures. Metrics are computed over successful executions.

V. Quantitative Results

Table I summarizes the successful-run statistics from the benchmark artifact. Values are computed over successful images; success rate is computed over all ten attempts.

A. Latency

Figure 2 shows the principal runtime separation. MobileSAM + LaMa completed successful cases in 16.63 s on average. EfficientSAM-S + LaMa required 25.74 s. Adding DINO raised the complete EfficientSAM pipeline above 61 s. Therefore, text grounding is the principal cost in the grounding-heavy configurations, while LaMa remains the major cost in the standalone pipelines.

B. Mask Quality

The mask-quality plot in Fig. 4 shows that YOLO26n-seg produced the strongest mean IoU and Dice among successful configurations. EfficientSAM-S followed with 0.827 IoU and 0.903 Dice. MobileSAM remained slightly lower at 0.810 IoU and 0.892 Dice while providing substantially lower segmentation latency.

C. Quality–Latency Relationship

Figure 5 emphasizes that increasing the model stack does not automatically improve the complete pipeline. The DINO + EfficientSAM configurations incur a large latency penalty while their mean IoU remains close to the standalone EfficientSAM

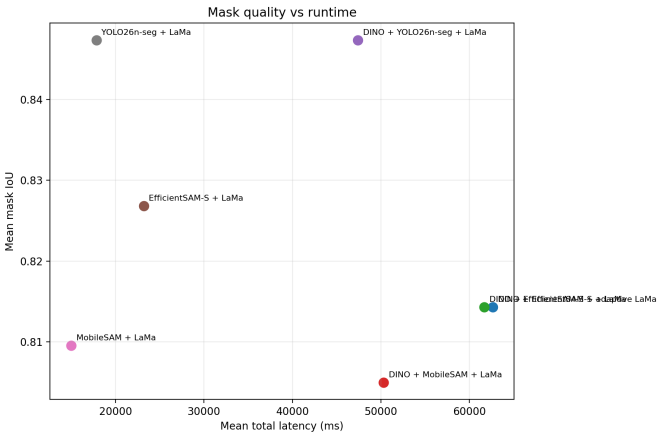


Fig. 5. Quality versus runtime across the tested architectures. The figure highlights the trade-off between mask fidelity and end-to-end latency.

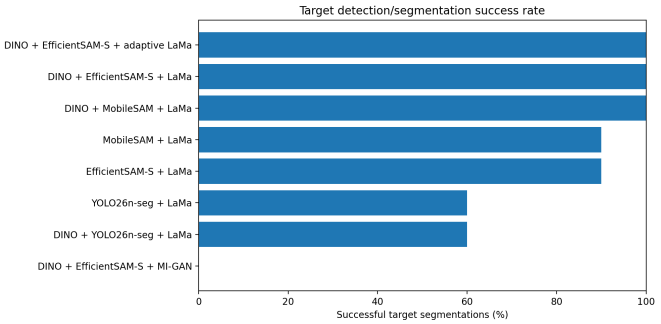


Fig. 6. Pipeline execution success rate over ten attempted images. The metric measures implementation completion, not perceptual quality.

result. Conversely, YOLO26n-seg provides strong successful-run masks but has a 60% execution success rate in the recorded implementation.

D. Execution Reliability

The success-rate plot in Fig. 6 exposes a major engineering consideration. DINO + MobileSAM and both DINO + EfficientSAM + LaMa configurations completed all ten attempts. MobileSAM + LaMa and EfficientSAM-S + LaMa completed nine. YOLO-based configurations completed six, while MI-GAN completed none because of the mask-rank interface error.

VI. Failure Analysis and Engineering Lessons

A. YOLO Mask/Image Shape Mismatch

The YOLO pipelines produced masks with dimensions that did not always match the original image dimensions. Direct compositing then triggered broadcasting errors. The correct engineering remedy is to preserve the source image dimensions and explicitly resize or warp the binary mask back to (H, W) using nearest-neighbor interpolation before RGB compositing. This failure illustrates that segmentation quality alone is insufficient: tensor geometry must be normalized at every pipeline boundary.

TABLE I
BENCHMARK RESULTS ACROSS THE EIGHT TESTED ARCHITECTURES. LATENCIES ARE IN MILLISECONDS AND ARE AVERAGED OVER SUCCESSFUL RUNS.

Architecture	Success	IoU	Dice	Seg. ms	Inpaint ms	Total ms	Change
YOLO26n-seg + LaMa	60%	0.847	0.916	3370	19884	23255	56.82
MobileSAM + LaMa	90%	0.810	0.892	4339	12292	16630	61.53
EfficientSAM-S + LaMa	90%	0.827	0.903	13293	12451	25744	61.68
DINO + YOLO26n-seg + LaMa	60%	0.847	0.916	33665	12391	46056	56.82
DINO + MobileSAM + LaMa	100%	0.805	0.885	38053	12265	50318	61.84
DINO + EfficientSAM-S + LaMa	100%	0.814	0.891	49851	12812	62663	60.86
DINO + EfficientSAM-S + adaptive LaMa	100%	0.814	0.891	49143	12550	61693	55.72
DINO + EfficientSAM-S + MI-GAN	0%	–	–	48849	–	–	–

B. No-Target Cases

MobileSAM, EfficientSAM-S, and YOLO-based pipelines each recorded no-target failures on at least one image. A production system should treat this as an explicit state rather than allowing downstream inpainting to run on an empty mask. Recommended handling is: validate mask area, validate bounding-box dimensions, return a structured failure, and optionally retry with a fallback segmenter.

C. MI-GAN ONNX Interface Failure

The MI-GAN branch consistently failed with an input-rank error: the mask supplied to ONNX Runtime had rank 3 while the model required rank 4. The required interface is therefore not simply an image-sized binary array; the adapter must create the expected batch/channel dimensions, typically $[1, 1, H, W]$. Because no valid MI-GAN output was produced, it was excluded from perceptual quality comparisons.

D. Grounding Cost

Grounding DINO-Tiny improved the semantic routing path but added tens of seconds to CPU-oriented execution. In the benchmark, DINO + EfficientSAM-S required approximately 49.85 s before LaMa, compared with 13.29 s for EfficientSAM-S alone. For edge deployment, this suggests a two-tier strategy: use a lightweight direct segmenter when the interaction already identifies a unique object, and invoke text grounding only when semantic disambiguation is required.

VII. VISUAL RESULTS

The output artifact contains per-image masks, boxes, segmented images, and removed images. Representative examples are shown in Figs. 7–10. These visualizations emphasize the practical difference between segmentation fidelity and the final reconstruction: a good mask can still yield poor removal if the inpainting model or mask boundary is incorrect.

VIII. LAMA OPTIMIZATION AND EDGE PROFILING

The benchmark artifact shows LaMa runtimes of roughly 12–20 s depending on the front-end architecture and implementation path. A later single-image test of the replacement pipeline measured 6.926 s for LaMa, 0.006 s for replacement compositing, and 6.932 s total. This measurement was not part of the 80-record benchmark and is therefore reported separately.

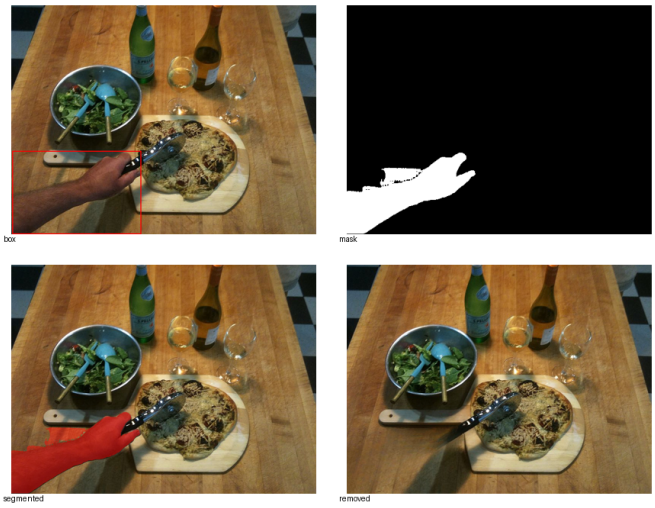


Fig. 7. Representative MobileSAM + LaMa result: box, mask, segmented region, and removed output for the same benchmark image.

The result is diagnostically important: replacement compositing is effectively negligible compared with reconstruction. The immediate optimization target is therefore LaMa inference. A crop-based strategy was subsequently explored in which only a padded bounding-box region is resized to the fixed LaMa input resolution, reconstructed, and pasted back into the original image. This reduces irrelevant computation while preserving the same ONNX interface.

The reference document also reports a quantized dilated LaMa deployment path, with FP16 and INT8 used to reduce CPU cost. [4], [9] The current experiments establish the engineering need for such optimization, but the 6.926 s single-image measurement should not be conflated with the reference document's separately reported mobile timings.

IX. OBJECT REPLACEMENT: EXPLORATION AND LIMITATIONS

Object replacement requires more than removing an object. The replacement region must be synthesized or inserted while respecting scene geometry, illumination, texture, and boundaries. The reference architecture uses Stable Diffusion 1.5 Inpainting with LCM sampling and LoRA conditioning for

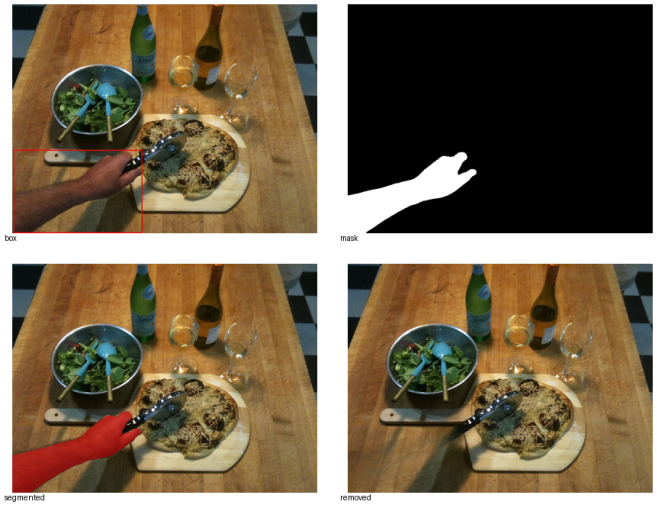


Fig. 8. Representative EfficientSAM-S + LaMa result.

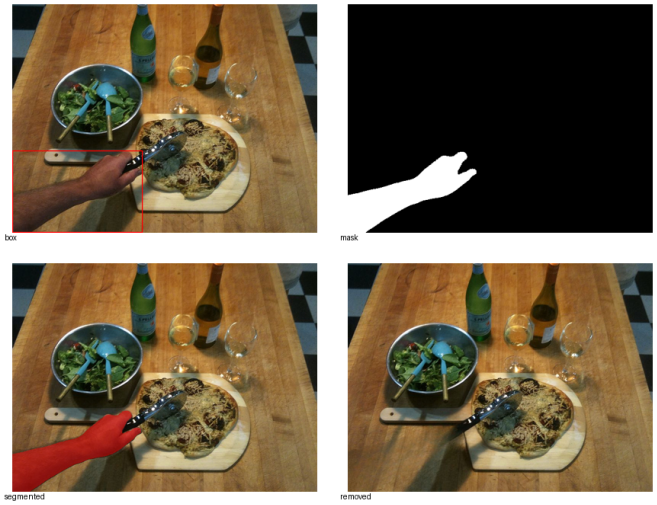


Fig. 9. Representative Grounding DINO-Tiny + EfficientSAM-S + LaMa result.

object and background replacement. It reports 5–8 diffusion steps and approximately 15 s on mobile-class CPUs in that design. [5]–[7], [9]

In the present implementation cycle, replacement was explored through an edge-friendly two-stage route: remove the original object with LaMa, then composite a replacement image into the recovered region with feathered blending. A single test measured 6.926 s for LaMa and only 0.006 s for compositing. This confirms that the compositing operation itself is inexpensive.

However, extensive replacement experiments were not completed because of time constraints. In particular, there was insufficient time to build a statistically meaningful benchmark across replacement categories, compare multiple generative backbones, tune prompts, and quantify perceptual consistency. Therefore, this report does not claim a validated replacement

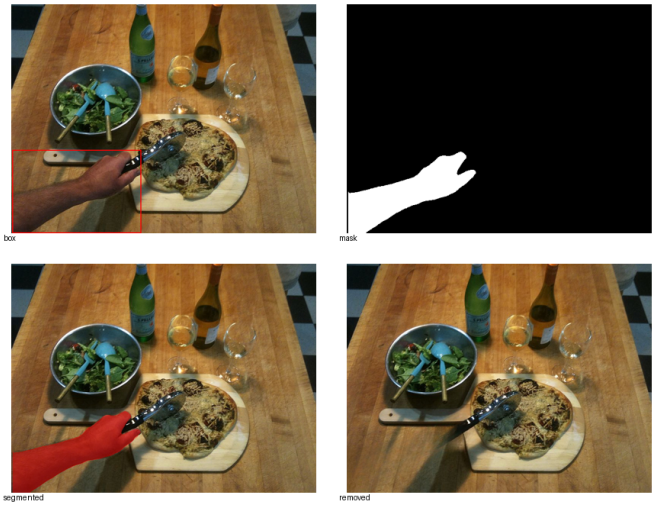


Fig. 10. Representative Grounding DINO-Tiny + YOLO26n-seg + LaMa result.

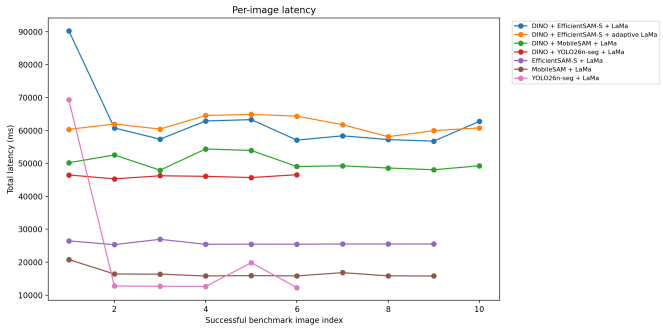


Fig. 11. Per-image latency variability recorded in the benchmark. Variability is influenced by segmentation behavior, image geometry, and reconstruction workload.

architecture. The replacement component should be treated as an exploratory extension rather than a completed benchmarked module.

X. ControlNet-Lite and Lightweight Generative Alternatives

ControlNet-Lite was investigated as a possible intermediate solution between deterministic LaMa reconstruction and full diffusion-based replacement. The principal motivation was to retain spatial control while reducing the computational burden of full ControlNet. During development, however, the exact checkpoint/configuration interface for the original ControlNet-Lite research implementation could not be validated sufficiently for a fair end-to-end benchmark. It was therefore not included as a numerical result in Table I.

This decision avoids presenting an unverified checkpoint substitution as an experimental result. Similarly, MI-GAN is listed with its failure status rather than an invented quality score. The broader lesson is that edge deployment requires model-interface reproducibility in addition to architectural suitability.

XI. Architecture Selection and Deployment Strategy

The experiments suggest a modular deployment policy rather than one universal model:

- **Fast default:** MobileSAM + LaMa when latency and reliability are primary constraints.
- **Higher mask fidelity:** EfficientSAM-S + LaMa when additional segmentation latency is acceptable.
- **High-quality segmentation with fallback:** YOLO26n-seg can be useful when its mask interface and no-detection handling are corrected.
- **Text-heavy editing:** Grounding DINO should be invoked selectively because it dominates CPU latency.
- **Generative replacement:** diffusion or controlled generative models should be treated as a separate, slower path rather than inserted into every removal request.

A practical edge system can therefore use an adaptive router: direct lightweight segmentation for unambiguous prompts, text grounding only for ambiguous requests, LaMa for deterministic removal, and a slower generative replacement branch only when the user explicitly requests synthesis.

XII. Reproducibility and Implementation Notes

The benchmark was run on a fixed ten-image COCO subset and stored as per-image records. The output artifact contains the architecture summary, all benchmark records, plots, and per-image visual outputs. The report uses the recorded successful-run means for latency and quality, while success rate is computed over all attempted images.

For reliable deployment, the following checks should be enforced before invoking an inpainting model:

1) image and mask dimensions must match;
2) mask rank must match the ONNX model contract;
3) mask area must exceed a minimum threshold;
4) bounding boxes must be inside image bounds;
5) empty detections must trigger a controlled fallback;
6) the output must be composited only inside the intended mask;
7) latency must be measured separately for preprocessing, segmentation, grounding, inpainting, and compositing.

XIII. Conclusion

This study evaluated eight concrete object-removal architectures under an edge-oriented CPU constraint. The experiments show that the central engineering trade-off is not simply accuracy versus model size. Reliability, tensor compatibility, grounding cost, and downstream reconstruction latency strongly affect whether a model can form a usable editing pipeline.

MobileSAM + LaMa provided the lowest mean successful latency among the reliable tested pipelines at 16.63 s, while EfficientSAM-S + LaMa increased mask quality to 0.827 IoU and 0.903 Dice at 25.74 s. YOLO26n-seg + LaMa produced the strongest successful-run mask scores, 0.847 IoU and 0.916 Dice, but only 60% of attempted runs completed because of mask-shape and detection failures. Adding Grounding DINO improved the semantic structure of the pipeline but increased latency dramatically. MI-GAN could not be evaluated because of an ONNX input-rank mismatch.

The subsequent LaMa profiling further clarified the optimization direction: a 6.926 s LaMa inference dominated a 0.006 s replacement-compositing stage. Cropped inference, quantization, and hardware-specific acceleration are therefore more valuable optimization targets than micro-optimizing compositing. Object replacement remains a promising extension, but its broader evaluation was constrained by available development time and should be treated as future work rather than as a completed experimental claim.

Overall, the experiments support a modular edge-editing architecture in which lightweight segmentation and deterministic inpainting form the reliable core, while expensive grounding and generative replacement are selectively activated according to user intent.

References

[1] A. Kirillov et al., "Segment Anything," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2023, pp. 4015–4026.

[2] C. Zhang et al., "Faster Segment Anything: Towards Lightweight SAM for Mobile Applications," arXiv:2306.14289, 2023.

[3] P. Vasu et al., "MobileCLIP: Fast Image-Text Models through Multi-Modal Reinforced Training," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2024, pp. 13866–13876.

[4] R. Suvorov et al., "Resolution-robust Large Mask Inpainting with Fourier Convolutions," in *Proc. IEEE/CVF Winter Conf. Appl. Comput. Vis. (WACV)*, 2022, pp. 2149–2159.

[5] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-Resolution Image Synthesis with Latent Diffusion Models," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2022, pp. 10684–10695.

[6] S. Luo et al., "Latent Consistency Models: Synthesizing High-Resolution Images with Few-Step Inference," arXiv:2310.04378, 2023.

[7] E. J. Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2022.

[8] R. Ranftl, K. Lasinger, D. Hafner, M. Schindler, and V. Koltun, "Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-shot Cross-dataset Transfer," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 3, pp. 1623–1637, 2022.

[9] Anonymous, "Reference Documentation for an Edge-Oriented Automated Photo Editing System," internal project document, pp. 1–7.